

# SPARE

Offline-first spare-parts operations, with auditable records and controlled sync.

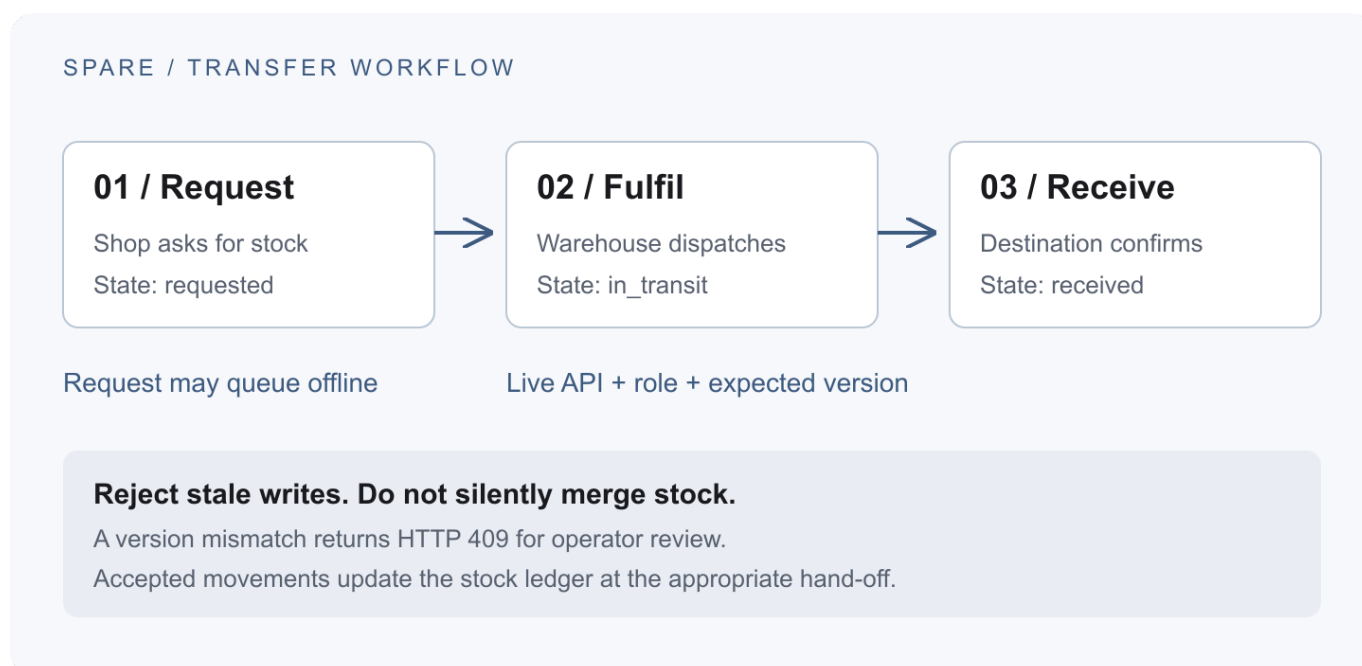
Project development: API, offline workflows and responsive console. Built as SPOP; presented here as SPARE. Local implementation verified; controlled-pilot preparation remains ongoing.

## The operational problem

One warehouse and multiple shops need to know what moved, who approved it and what still awaits sync. SPARE connects stock receipts, sales recording, transfers, purchasing, approvals, accounting and branch reports. It is an operations workspace, not a payment-processing POS.

## Separate the physical hand-offs

Transfer requests may enter a Dexie outbox during a connection failure. Fulfilment and receipt must contact the API, where role and expected-version checks prevent unauthorized or stale transitions. Reconnect replay reuses client request IDs; a version mismatch returns HTTP 409 for operator review.

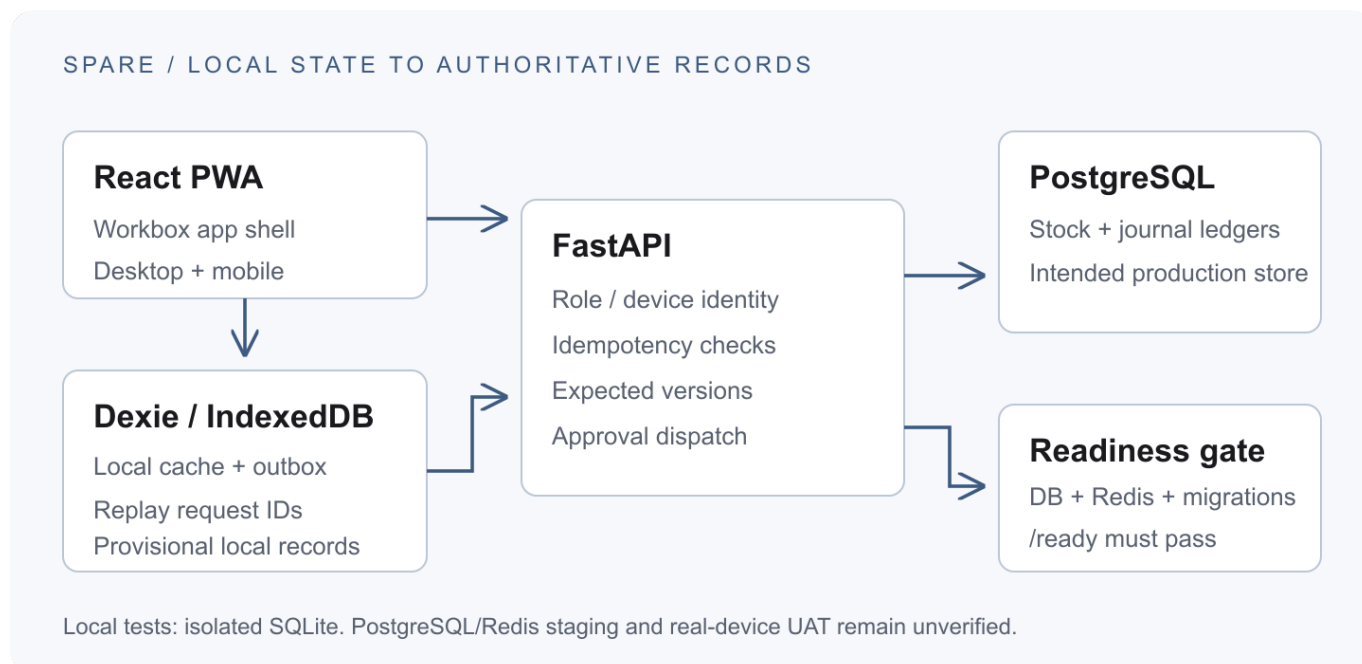


Original workflow diagram. Accepted movements enter the stock ledger at the appropriate hand-off; local queued records remain provisional.

## A deliberate offline boundary

Receipts, sales and transfer requests have local queue paths; stock hand-off actions stay online. The current retry policy is bounded, and conflicts are surfaced rather than silently merged. Client error classification and recovery ergonomics still need real-device pilot review.

## Architecture and engineering choices



Original architecture diagram. React/TypeScript and Workbox provide the PWA shell; Dexie keeps the cache/outbox. FastAPI enforces six fixed roles, JWT/device identity, idempotency and selected version checks. PostgreSQL is intended for production; local tests use isolated SQLite.

## Records before dashboards

Stock and accounting use append-only ledgers. Shared approval dispatch keeps purchase and expense approvals aligned with the records they gate; reports derive totals from recorded movements and journal entries. The trade-off is careful transaction and reconciliation testing.

## Explainable replenishment, not ML

Suggestions use trailing 30-day sales velocity and current stock. This transparent heuristic is inspectable before production data exists for model validation. Trained demand forecasting remains deferred; no ML accuracy or measured commercial savings are claimed.

## Verified locally, bounded honestly

Rerun on 5 October 2026: 75 backend tests and 7 frontend tests passed. Coverage includes permissions, operational transitions, approvals/accounting, idempotency, stale-version recovery, readiness and migrations, plus keyboard/focus, mobile navigation and local offline replay.

Still outstanding: staging PostgreSQL/Redis readiness, real-device UAT, backup/restore rehearsal and business-outcome measurement. The earlier local /ready check returned 503 when dependencies were incomplete; a production /ready response must be 200 before pilot traffic.

Next: validate the full staging gate, exercise offline hand-offs on intended devices, and reconcile stock and journal totals before a controlled pilot.

Source: SPOP implementation and pilot verification/UAT records, reviewed 5 October 2026; test suites rerun that day. This summary publishes no client records, credentials, logs or full internal documentation.